

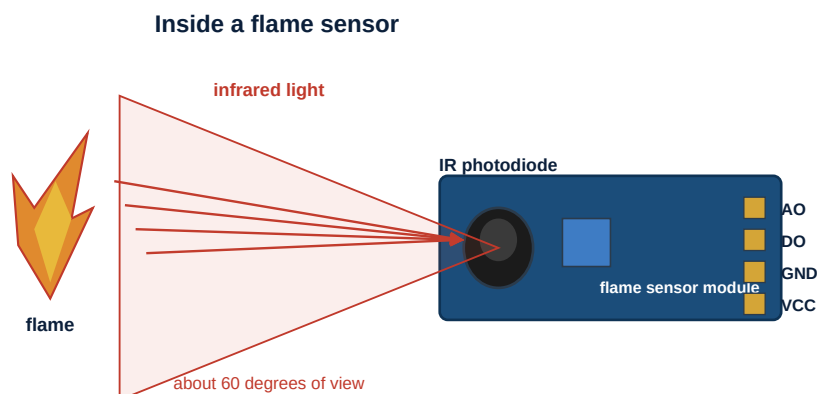
Inside a Flame Sensor

Today you will

- Learn what a flame sensor actually detects.
- Understand why a second sensor is needed to confirm.
- Take your first readings from a supervised candle.

The obvious way to detect a fire is to measure temperature. It is also the wrong way. Air is a poor conductor of heat, so a thermometer a metre from a flame may take minutes to notice anything - and in a fire, minutes are everything.

A flame sensor takes a completely different approach. It does not measure heat at all. It watches for **light** - specifically infrared light in the band from about 760 to 1100 nanometres, which is exactly the band a burning flame emits strongly. Light travels at 300,000 kilometres per second, so the sensor reacts in milliseconds.



Why it is not a heat sensor

A flame gives off infrared light at around 760 to 1100 nanometres. The black dome is a photodiode tuned to exactly that band, so it is nearly blind to ordinary room light. It therefore reacts in milliseconds - long before the air around it has warmed up at all. But sunlight and filament bulbs also emit infrared, which is why a second sensor is used to confirm before an alarm is raised.

Figure 3.1 - It sees the light of a fire, not the heat of one

Behind the Scenes

The black dome is a **photodiode**, and the black plastic is not decoration - it is an optical filter. It blocks visible light while letting infrared through, which is why the sensor barely reacts to a torch shone straight at it. The same black plastic is used on your TV remote's receiver, for exactly the same reason.

But there is a catch, and it is the reason this project needs two sensors. Sunlight is full of infrared. So is a filament bulb, and so is a hot soldering iron. A flame sensor on its own will happily announce a fire because somebody opened a curtain. The fix is **confirmation**: require a second, independent measurement to agree before acting.

Sensor	What it sees	How fast	Foiled by
Flame sensor	Infrared light from a flame	milliseconds	sunlight, filament bulbs, hot metal
DHT11	Air temperature rising	tens of seconds	a warm day, a heater, your hand
Both together	IR light AND rising temperature	seconds	almost nothing in a classroom

Try This

Wire the flame sensor: VCC to 5V, GND to GND, AO to A0, DO to D2. Print the AO value on the Serial Monitor. Note the number in a normal room, then near a window, then with a torch pointed at it, and finally with a supervised candle at 30 cm. Those four numbers tell you where your threshold must sit.

SESSION 7

one class

Building the Alarm

Today you will

- Wire both sensors, the buzzer, the lamps and the reset switch.
- Understand why a switch needs a pull-down resistor.
- Get a first working alarm running.

Before wiring, it is worth knowing what your confirming sensor actually contains. The DHT11 is two instruments and a small computer in one blue box.

Inside the DHT11	How it works	What limits it
Humidity sensor	Two plates with a moisture-absorbing film between them. As the film takes in water, its ability to store charge changes, and the chip measures that change.	Only whole percentages, plus or minus 5
Temperature sensor	A thermistor - a resistor whose resistance falls sharply as it warms. The chip compares it against a fixed reference.	Whole degrees, plus or minus 2
Small controller	Converts both readings and sends all forty bits as one precisely timed train of pulses down the single DATA wire.	One reading every two seconds

Behind the Scenes

That single DATA wire is doing something remarkable. The Arduino pulls it low to ask for a reading; the DHT11 replies with forty pulses where a short pulse means 0 and a long one means 1. Getting the timing right is fiddly to the microsecond, which is exactly why a library exists. It also explains the two-second rule: the sensor genuinely needs that long to take a fresh measurement.

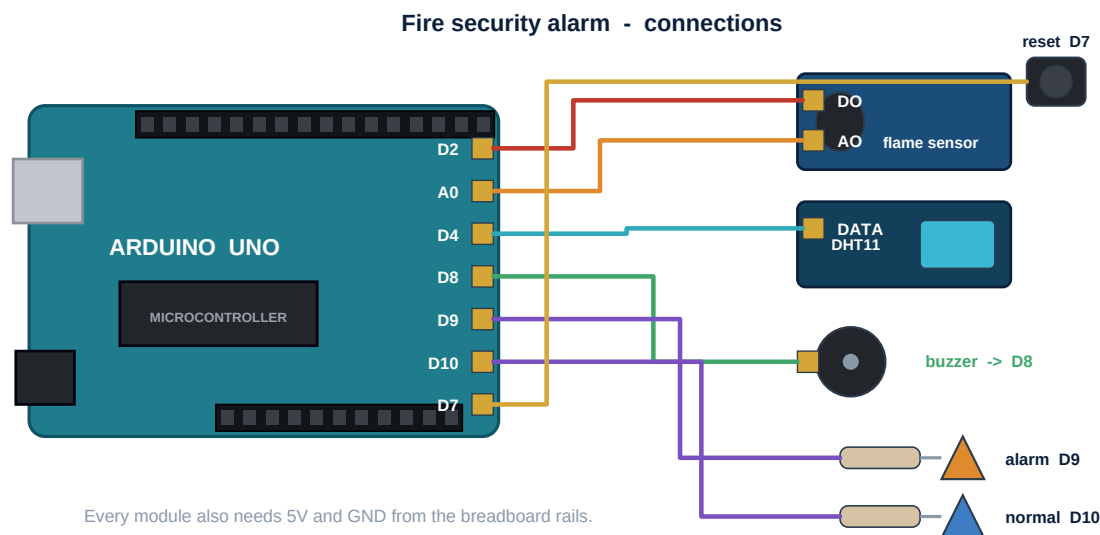


Figure 3.2 - Two sensors in, three alerts out, one reset

Component pin	Arduino pin	Why
Flame sensor DO	D2	A quick yes-or-no trip
Flame sensor AO	A0	How strong the infrared is
DHT11 DATA	D4	Temperature, for confirmation
Buzzer +	D8	The alarm
Orange LED long leg	D9 through 220 ohm	Alarm lamp
Blue LED long leg	D10 through 220 ohm	Normal lamp

Component pin	Arduino pin	Why
Push switch, one side	5V	Supply side
Push switch, other side	D7, and 10K to GND	Reset input with a pull-down

Behind the Scenes

Why does the switch need that 10K resistor? Because an unconnected input pin is not at 0V - it is **floating**, picking up stray electrical noise from the air and the wires, and reading HIGH and LOW at random. The 10K resistor gently ties the pin to ground so it reads a firm LOW, while still being weak enough that pressing the switch easily overpowers it and pulls the pin to 5V. Ten thousand ohms is the standard compromise.

7.1 Behind the Scenes: How a Buzzer Makes Sound

Prise the sticker off a buzzer and you would find a thin brass disc with a layer of ceramic bonded to it. That ceramic is **piezoelectric**: apply a voltage and it physically changes shape, bending the disc a fraction of a millimetre. Remove the voltage and it springs back.

Do that thousands of times a second and the disc pushes the air thousands of times a second - and that is sound. Your kit contains an **active** buzzer, which has its own oscillator built in, so a plain `digitalWrite(buzzer, HIGH)` produces a note. A **passive** buzzer has no oscillator, and you would have to supply the frequency yourself with `tone(pin, 2000)`.

	Active buzzer	Passive buzzer
How you drive it	<code>digitalWrite HIGH</code>	<code>tone(pin, frequency)</code>
Pitch	fixed by the maker	any pitch you choose
Best for	alarms and alerts	melodies and warning patterns

7.2 Behind the Scenes: Why a Switch Bounces

A push switch feels like a clean action to your finger, but electrically it is not. The two metal contacts are springy, and when they meet they physically bounce apart and back together several times in the first few milliseconds. To a chip running sixteen million operations a second, one press can therefore look like five or six separate presses.

The cure is called **debouncing**. The simplest version is the one in your alarm code: after detecting a press, wait a few hundred milliseconds before looking again, by which time the chatter has finished. It is crude but entirely effective for a reset button.

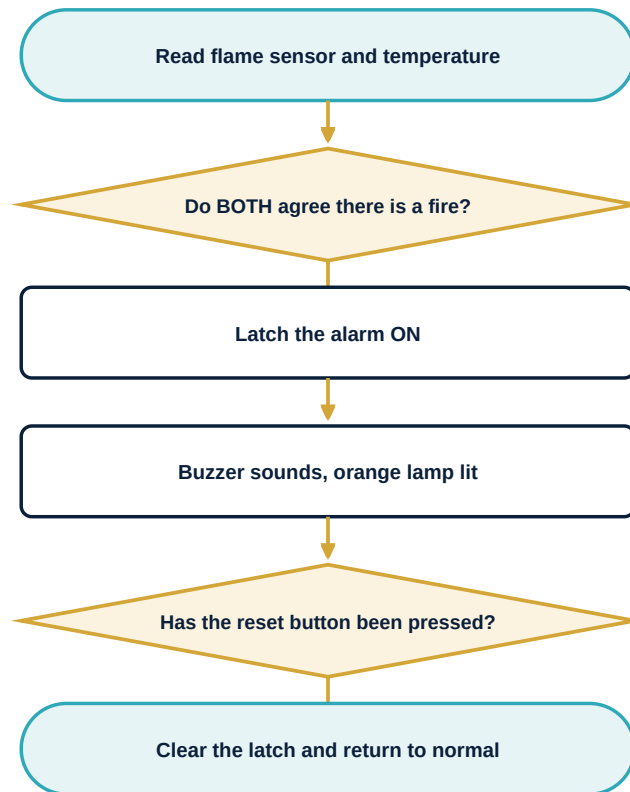


Figure 3.3 - Confirmation, then latching, then a human reset

SESSION 8

one class

Latching, Resetting and False Alarms

Today you will

- Understand why an alarm must latch.
- Write the full program with confirmation and reset.
- Test for false positives and false negatives.

Here is a requirement that changes everything. Suppose a flame flares for half a second and then goes out. A simple if statement would sound the buzzer for that half second and then fall silent, and nobody would ever know. A real alarm must **latch**: once triggered, it stays on until a person deliberately clears it.

Latching is done with a **boolean** - a variable that holds only true or false. The sensors can set it to true, but only the reset button can set it back to false. That single idea turns a reaction into a memory.

FireSecurityAlarm.ino

```

#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

int flameDO = 2, flameAO = A0;
int buzzer = 8, alarmLed = 9, normalLed = 10, resetBtn = 7;

int flameLimit = 300; // below this on A0 means strong infrared
float tempLimit = 38.0; // degrees Celsius - set from your own data
  
```

```

boolean alarmLatched = false; // this is the machine's memory
int alarmCount = 0;

void setup() {
  pinMode(flameDO, INPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(alarmLed, OUTPUT);
  pinMode(normalLed, OUTPUT);
  pinMode(resetBtn, INPUT);
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  int ir = analogRead(flameAO);
  float t = dht.readTemperature();

  if (isnan(t)) { t = 0; } // sensor did not answer this time

  boolean seesFlame = (ir < flameLimit);
  boolean isHot = (t > templimit);

  Serial.print("IR: "); Serial.print(ir);
  Serial.print(" T: "); Serial.print(t);
  Serial.print(" latched: "); Serial.println(alarmLatched);

  if (seesFlame && isHot) { // BOTH must agree
    if (!alarmLatched) { // only count a new alarm once
      alarmCount = alarmCount + 1;
    }
    alarmLatched = true;
  }

  if (digitalRead(resetBtn) == HIGH) {
    alarmLatched = false; // only a human can clear it
    delay(300); // simple debounce
  }

  if (alarmLatched) {
    digitalWrite(alarmLed, HIGH);
    digitalWrite(normalLed, LOW);
    digitalWrite(buzzer, HIGH);
    delay(150);
    digitalWrite(buzzer, LOW);
    delay(150);
  } else {
    digitalWrite(alarmLed, LOW);
    digitalWrite(normalLed, HIGH);
    digitalWrite(buzzer, LOW);
    delay(300);
  }
}

```

Read the two lines that create `seesFlame` and `isHot`. A comparison such as `ir < flameLimit` is already true or false, so it can be stored straight into a boolean. Naming those two conditions makes the important line - `if (seesFlame && isHot)` - read almost like English. Code that reads like its own explanation is code you can still understand next month.

8.1 Testing Honestly

Every detector in the world can be wrong in two different ways, and engineers name them separately because the consequences are so different.

Name	What happens	Why it matters
False positive	Alarm sounds when there is no fire	People stop trusting the alarm and eventually disconnect it
False negative	There is a fire and the alarm stays silent	The alarm has failed at the only job it had

Test	Should the alarm fire?	Did it?	Tick
Normal classroom, nothing happening	No	_____	■
Bright torch shone at the sensor	No	_____	■
Sunlight through an open window	No	_____	■
Warm hand cupped over the DHT11	No	_____	■
Supervised candle at 30 cm	Yes, within 2 seconds	_____	■
Candle removed	Stays latched	_____	■
Reset button pressed	Returns to normal	_____	■

Challenge

Add a second stage to your alarm. If the flame sensor alone trips, show a flashing orange lamp as a *warning* but stay silent. Only when the temperature confirms should the buzzer sound. Real industrial detectors work exactly this way, and it is called a two-stage alarm.

New word	What it means
Photodiode	A component that produces a current when light falls on it.
Infrared	Light just beyond red, invisible to us, given off strongly by flames.
Latching	Staying on once triggered, until deliberately cleared.
Boolean	A variable that holds only true or false.
Floating input	An unconnected pin, reading randomly because of stray noise.
False positive / negative	Alarming with no fire / staying silent during one.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 3**A. Choose the correct answer**

1. A flame sensor detects a fire by sensing
(a) heat (b) smoke (c) infrared light (d) sound
2. The black dome on the sensor acts as
(a) a lens (b) an optical filter (c) a heat sink (d) decoration
3. Two sensors are used together in order to
(a) measure faster (b) reduce false alarms (c) save power (d) look impressive
4. A latching alarm stays on until
(a) the fire goes out (b) a human resets it (c) the power dips (d) 10 seconds pass
5. A pin with nothing connected to it is described as
(a) grounded (b) floating (c) latched (d) powered
6. An alarm that stays silent during a real fire is a
(a) false positive (b) false negative (c) latch (d) debounce

B. Fill in the blanks

- 1 A flame emits infrared strongly between about _____ and _____ nanometres.
- 2 A variable holding only true or false is called a _____.
- 3 The switch input needs a _____ resistor to stop it floating.
- 4 An alarm that sounds with no fire present is a false _____.

C. Answer in one or two lines

- 1 Explain why measuring temperature alone is a poor way to detect fire quickly.
- 2 Write the IR and temperature values you measured, and the limits you chose.
- 3 Why is a false positive dangerous even though nobody is hurt by it?
- 4 Explain in your own words what latching does and why it is needed.

UNIT 4

Intermediate Project: Smart 16x2 LCD Weather System

Sessions 9, 10 and 11

Until now, seeing your data has meant staring at a laptop. This unit cuts the cord. Your machine gets a screen of its own, and with it the ability to stand alone on a shelf and be useful to somebody who has never seen your code.

By the end of this unit you will be able to:

- Explain how a liquid crystal display forms a character.
- Wire a parallel display in 4-bit mode and explain each control line.
- Present multi-sensor data in a readable, human-facing format.
- Track minimum and maximum values over time.

SESSION 9

one class

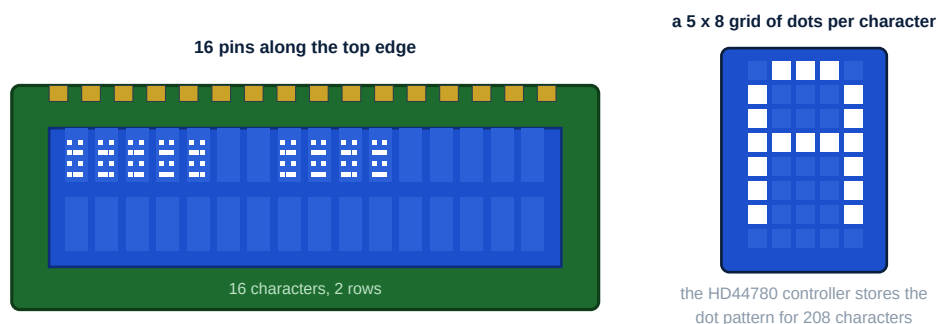
Inside a Liquid Crystal Display

Today you will

- Learn how liquid crystal actually works.
- Understand what the HD44780 controller does for you.
- Identify all sixteen pins on the display.

The name sounds like a contradiction. How can something be both liquid and crystal? The answer is that liquid crystal is a genuine fourth state of matter. Its molecules flow freely like a liquid, but they also line up with one another like a crystal - and crucially, that alignment can be changed by a small voltage.

Inside a 16x2 character LCD



Why it is called liquid crystal

Between two sheets of glass sits a liquid whose long molecules normally line up in a twist. That twist rotates light passing through, so it escapes the far filter and the pixel looks pale. Apply a small voltage and the molecules straighten. The light no longer twists, the far filter blocks it, and that dot goes dark. No dot makes its own light - the backlight behind does that, which is why an LCD needs its LED backlight and why the contrast pot changes what you can read.

Behind the Scenes

Notice what this means: an LCD dot produces **no light of its own**. It only blocks or passes the light of the backlight behind it. That is why the display is unreadable if the backlight fails, why the contrast potentiometer matters so much, and why an LCD uses so little power compared with a screen that generates its own light. It is also why a calculator screen still works in bright sun and a phone screen struggles.

Driving thousands of individual dots would be impossible from an Arduino. Fortunately the black blob on the back of your display is a dedicated controller chip, the **HD44780**. It has been the industry standard since the 1980s and it does a great deal for you: it stores the dot pattern for 208 characters, keeps track of where the cursor is, and refreshes every dot continuously. You simply send it a letter, and it draws it.

Pin	Name	What it is for
1, 2	VSS, VDD	Ground and 5V power
3	V0	Contrast. This is what the potentiometer sets.
4	RS	Register Select: is this byte a character, or a command?
5	RW	Read or Write. Tied to ground - we only ever write.
6	E	Enable. A pulse here means 'take the data now'.
7 to 14	D0 to D7	The data lines. In 4-bit mode we use only D4 to D7.

Pin	Name	What it is for
15, 16	A, K	The LED backlight. A goes to 5V through a 220 ohm resistor.

Behind the Scenes

Why use only four of the eight data lines? Because eight would eat eight Arduino pins and leave almost nothing for your sensors. In **4-bit mode** each byte is sent as two halves, one after the other. It is half the speed - and completely irrelevant, because the display only needs updating a few times a second while the chip runs sixteen million times faster.

SESSION 10

one class

Wiring and Driving the Display

Today you will

- Wire the LCD in 4-bit mode with the contrast pot.
- Get your group name on the screen.
- Learn setCursor and how the two rows are addressed.

LCD weather system - connections

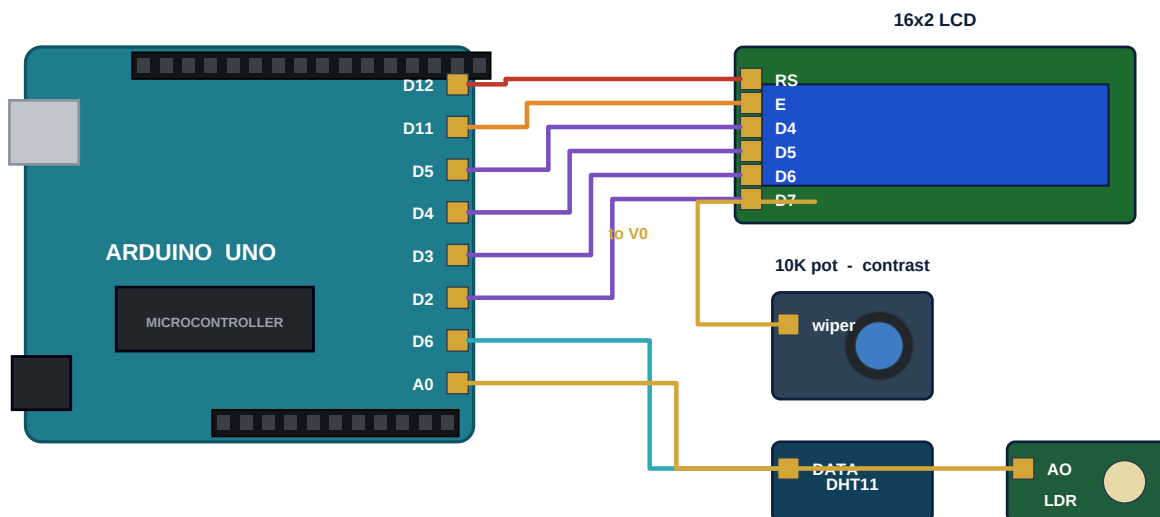


Figure 4.2 - Six control wires run the display in 4-bit mode

LCD pin	Goes to	Note
VSS	GND	
VDD	5V	
V0	Potentiometer wiper	Turn it until the boxes just appear, then back slightly
RS	D12	Command or character
RW	GND	Always write
E	D11	The 'take it now' pulse
D4, D5, D6, D7	D5, D4, D3, D2	Note the deliberate reversal - follow it exactly
A	5V through 220 ohm	Backlight positive
K	GND	Backlight negative

LcdHello.ino

```
#include <LiquidCrystal.h>

// the numbers, in order: RS, E, D4, D5, D6, D7
```

```

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  lcd.begin(16, 2);           // 16 columns, 2 rows
  lcd.setCursor(0, 0);        // column 0, row 0
  lcd.print("PIXIU TECH");
  lcd.setCursor(0, 1);        // column 0, row 1
  lcd.print("Class 9 Lab");
}

void loop() {
}

```

Watch Out

If you see a row of solid white blocks and nothing else, your wiring is probably fine and your **contrast** is wrong - turn the potentiometer slowly. If you see nothing at all, not even blocks, check the backlight and VDD first. Learning to tell those two symptoms apart will save you an hour.

One detail about `setCursor(column, row)` catches everybody: it takes **column first, then row**, and both count from zero. So the last character of the bottom line is `setCursor(15, 1)`. Get the order backwards and your text vanishes off the edge of a screen that only has two rows.

SESSION 11

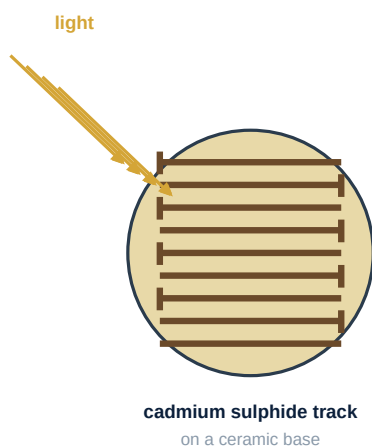
one class

Adding the Sensors and Rotating Screens

Today you will

- Feed the DHT11 and the LDR onto the display.
- Rotate between two screens without freezing the program.
- Track the highest and lowest temperature of the day.

Your third reading comes from the LDR, and it is worth a minute to see what is inside it. Under the clear window sits a zig-zag track of **cadmium sulphide** printed on a ceramic base. The zig-zag is not decoration: it makes the track long and narrow, which magnifies the effect you are about to measure.



What the light actually does

In the dark

electrons stay locked in place
resistance high, about 1,000,000 ohms

In bright light

photons knock electrons free
resistance low, a few hundred ohms

This is called photoconductivity. Notice what the sensor does not do: it never measures brightness in any unit. It only changes its resistance, and the module turns that into a voltage your Arduino can read.

Figure - Inside an LDR: light knocks electrons loose and current flows more easily

Behind the Scenes

In darkness the electrons in that material are bound in place, so almost nothing flows and the resistance can reach a million ohms. When light lands, each photon carries just enough energy to knock an electron free, and every freed electron makes the material a little more conductive. In bright sun the resistance can fall to a few hundred ohms - a change of several thousand times. The effect is called **photoconductivity**, and the same principle runs the light sensor that dims your phone screen.

Notice what the LDR does **not** do. It never reports brightness in any unit. It only changes its resistance, and the module turns that into a voltage that your ADC can read. What that number means is entirely for you to decide - which is exactly the job of a threshold.

Three readings will not fit comfortably on thirty-two characters, so your display must take turns. The obvious way is a long `delay()` - and it is the wrong way, because during a delay the Arduino is completely frozen and cannot read a sensor or respond to anything.

The professional approach uses `millis()`, which reports how many milliseconds the board has been running. Instead of saying *stop here for three seconds*, you say *if three seconds have passed since last time, switch the screen*. The program never stops, and it can still do other work in between.

LcdWeatherSystem.ino

```
#include <LiquidCrystal.h>
#include <DHT.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
#define DHTPIN 6
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

int ldrPin = A0;

float tMin = 99.0, tMax = -99.0; // start deliberately out of range

unsigned long lastSwap = 0;
int screen = 0;

void setup() {
  lcd.begin(16, 2);
  dht.begin();
  Serial.begin(9600);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  int light = analogRead(ldrPin);

  if (!isnan(t)) { // only record a real reading
    if (t < tMin) { tMin = t; }
    if (t > tMax) { tMax = t; }
  }

  if (millis() - lastSwap > 3000) { // three seconds have passed
    lastSwap = millis();
    screen = 1 - screen; // flip between 0 and 1
    lcd.clear();
  }

  if (screen == 0) {
    lcd.setCursor(0, 0);
    lcd.print("Temp: "); lcd.print(t); lcd.print(" C");
    lcd.setCursor(0, 1);
    lcd.print("Humid: "); lcd.print(h); lcd.print(" %");
  }
}
```

```

    } else {
      lcd.setCursor(0, 0);
      lcd.print("Light: "); lcd.print(light);
      lcd.setCursor(0, 1);
      lcd.print("L:"); lcd.print(tMin);
      lcd.print(" H:"); lcd.print(tMax);
    }

    delay(500);
  }
}

```

Behind the Scenes

Look at `screen = 1 - screen;`. If `screen` is 0 it becomes 1; if it is 1 it becomes 0. One line of arithmetic replaces an entire if-else. Small tricks like this are everywhere in professional code - though remember that clever code is only good code when it is still readable.

11.1 Making Your Own Character

The HD44780 stores 208 ready-made characters, but it also keeps eight slots empty for you. A character is a 5 by 8 grid, so you can describe one as eight rows of five bits. Here is a degree symbol and a small thermometer.

CustomCharacter.ino

```

byte degree[8] = {
  0b00110,
  0b01001,
  0b01001,
  0b00110,
  0b00000,
  0b00000,
  0b00000,
  0b00000
};

void setup() {
  lcd.begin(16, 2);
  lcd.createChar(0, degree); // store it in slot 0
  lcd.setCursor(0, 0);
  lcd.print("Temp 28");
  lcd.write(byte(0));        // draw your own character
  lcd.print("C");
}

```

Read the ones and zeros as a picture: each 1 is a dot that lights, each 0 stays dark. Draw your design on squared paper first, then write down each row as five binary digits. The 0b at the front simply tells the compiler that what follows is binary rather than an ordinary number.

Challenge

Add a comfort message on the second row: DRY below 40 percent humidity, OK between 40 and 65, HUMID above 65. Then add a third screen showing how long the system has been running, using `millis()` / 60000 to get minutes.

New word	What it means
Liquid crystal	A state of matter that flows like a liquid but aligns like a crystal.
Backlight	The LED behind an LCD that supplies all the light you see.
HD44780	The standard controller chip on the back of a character LCD.
4-bit mode	Sending each byte as two halves, to save Arduino pins.
millis()	How long the board has been running, in milliseconds.
Non-blocking	Code that waits without freezing everything else.